

HDOC Day-41

Menu-Driven Paragraph Operations Using Strings and Pointers in C

Algorithm

1. Read paragraph 1 and paragraph 2 using fgets() in readParagraph().
2. Display the menu repeatedly until Exit is selected.
3. Choice 1: copy each paragraph, tokenize it with strtok(), remove punctuation from tokens, and compare word lengths to obtain the longest and shortest words.
4. Choice 2: scan each paragraph character by character and copy only one space for a run of whitespace characters.
5. Choice 3: scan each paragraph; uppercase the first alphabetic character of each word and lowercase the remaining alphabetic characters.
6. Choice 4: tokenize each paragraph, convert words to lowercase, compare them using strcmp(), and count the frequency of every distinct word.
7. Choice 5: display the original paragraphs again.
8. Choice 6: exit. For any other choice, display an invalid-choice message.

Test Case Table

TC	Choice	Input	Expected Result
1	1	P1: This is a simple paragraph. / P2: C programming is fun.	P1 longest=paragraph, shortest=a; P2 longest=programming, shortest=C
2	2	P1: This is C. / P2: Hello world.	Repeated spaces removed
3	3	P1: hello WORLD / P2: c PROGRAMMING	Hello World / C Programming
4	4	P1: C is fun and C is useful. / P2: Code code TEST test.	Case-insensitive frequencies: c=2, is=2...; code=2, test=2
5	1	P1 and P2 empty	Longest=(none), Shortest=(none)
6	9	Invalid menu choice	Invalid choice. Please enter 1 to 6.
7	6	Exit	Exiting program.

C Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAX 500
#define MAX_WORDS 100
#define WORD_LEN 60
```

```
void readParagraph(char paragraph[], size_t size);
```

```

void findLongestAndShortestWords(const char paragraph[], char longestWord[], char shortestWord[]);
void removeRepeatedSpaces(const char paragraph[], char modifiedParagraph[]);
void convertToTitleCase(const char paragraph[], char titleCaseParagraph[]);
void findWordFrequencies(const char paragraph[]);
void cleanWord(const char source[], char destination[]);

```

```

int main(void)
{
    char p1[MAX], p2[MAX];
    char longest[WORD_LEN], shortest[WORD_LEN];
    char modified[MAX], title[MAX];
    int choice;

    printf("Enter paragraph 1:\n");
    readParagraph(p1, MAX);
    printf("Enter paragraph 2:\n");
    readParagraph(p2, MAX);

    do {
        printf("\n--- PARAGRAPH OPERATIONS MENU ---\n");
        printf("1. Find longest and shortest words\n");
        printf("2. Remove repeated consecutive spaces\n");
        printf("3. Convert paragraphs to title case\n");
        printf("4. Find word frequencies\n");
        printf("5. Display original paragraphs\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        getchar();

        switch (choice) {
            case 1:
                findLongestAndShortestWords(p1, longest, shortest);
                printf("Paragraph 1 - Longest: %s, Shortest: %s\n", longest, shortest);
                findLongestAndShortestWords(p2, longest, shortest);
                printf("Paragraph 2 - Longest: %s, Shortest: %s\n", longest, shortest);
                break;
            case 2:
                removeRepeatedSpaces(p1, modified);
                printf("Paragraph 1: %s\n", modified);
                removeRepeatedSpaces(p2, modified);
                printf("Paragraph 2: %s\n", modified);
                break;
            case 3:
                convertToTitleCase(p1, title);
                printf("Paragraph 1: %s\n", title);
                convertToTitleCase(p2, title);
                printf("Paragraph 2: %s\n", title);
                break;
            case 4:
                printf("Paragraph 1 word frequencies:\n");
                findWordFrequencies(p1);
                printf("Paragraph 2 word frequencies:\n");
                findWordFrequencies(p2);
                break;
            case 5:
                printf("Paragraph 1: %s\n", p1);
                printf("Paragraph 2: %s\n", p2);
                break;
            case 6:
                printf("Exiting program.\n");
                break;
            default:

```

```

        printf("Invalid choice. Please enter 1 to 6.\n");
    }
} while (choice != 6);

return 0;
}

void readParagraph(char paragraph[], size_t size)
{
    size_t len;
    fgets(paragraph, size, stdin);
    len = strlen(paragraph);
    if (len > 0 && paragraph[len - 1] == '\n')
        paragraph[len - 1] = '\0';
}

void cleanWord(const char source[], char destination[])
{
    int i, j = 0;
    for (i = 0; source[i] != '\0' && j < WORD_LEN - 1; i++) {
        if (isalnum((unsigned char)source[i]))
            destination[j++] = source[i];
    }
    destination[j] = '\0';
}

void findLongestAndShortestWords(const char paragraph[], char longestWord[], char shortestWord[])
{
    char copy[MAX], *token, word[WORD_LEN];
    size_t maxLen = 0, minLen = (size_t)-1, len;
    longestWord[0] = shortestWord[0] = '\0';
    strcpy(copy, paragraph);
    token = strtok(copy, " \t");
    while (token != NULL) {
        cleanWord(token, word);
        len = strlen(word);
        if (len > 0) {
            if (len > maxLen) { maxLen = len; strcpy(longestWord, word); }
            if (len < minLen) { minLen = len; strcpy(shortestWord, word); }
        }
        token = strtok(NULL, " \t");
    }
    if (maxLen == 0) {
        strcpy(longestWord, "(none)");
        strcpy(shortestWord, "(none)");
    }
}

void removeRepeatedSpaces(const char paragraph[], char modifiedParagraph[])
{
    int i = 0, j = 0, previousSpace = 0;
    while (paragraph[i] != '\0') {
        if (isspace((unsigned char)paragraph[i])) {
            if (!previousSpace && j > 0) {
                modifiedParagraph[j++] = ' ';
                previousSpace = 1;
            }
        } else {
            modifiedParagraph[j++] = paragraph[i];
            previousSpace = 0;
        }
        i++;
    }
}

```

```

        if (j > 0 && modifiedParagraph[j - 1] == ' ') j--;
        modifiedParagraph[j] = '\0';
    }

void convertToTitleCase(const char paragraph[], char titleCaseParagraph[])
{
    int i = 0, newWord = 1;
    while (paragraph[i] != '\0') {
        unsigned char ch = (unsigned char)paragraph[i];
        if (isalpha(ch)) {
            titleCaseParagraph[i] = newWord ? (char)toupper(ch) : (char)tolower(ch);
            newWord = 0;
        } else {
            titleCaseParagraph[i] = paragraph[i];
            if (isspace(ch)) newWord = 1;
        }
        i++;
    }
    titleCaseParagraph[i] = '\0';
}

void findWordFrequencies(const char paragraph[])
{
    char copy[MAX], words[MAX_WORDS][WORD_LEN], *token, word[WORD_LEN];
    int counts[MAX_WORDS] = {0}, unique = 0, i, found;
    strcpy(copy, paragraph);
    token = strtok(copy, " \\t");
    while (token != NULL) {
        cleanWord(token, word);
        for (i = 0; word[i] != '\0'; i++) word[i] = (char)tolower((unsigned char)word[i]);
        if (word[0] != '\0') {
            found = -1;
            for (i = 0; i < unique; i++)
                if (strcmp(words[i], word) == 0) { found = i; break; }
            if (found >= 0) counts[found]++;
            else if (unique < MAX_WORDS) {
                strcpy(words[unique], word);
                counts[unique] = 1;
                unique++;
            }
        }
        token = strtok(NULL, " \\t");
    }
    if (unique == 0) printf("No words found.\n");
    else for (i = 0; i < unique; i++) printf("%s = %d\n", words[i], counts[i]);
}

```

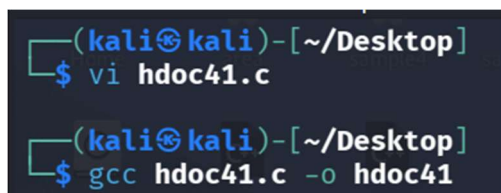
Compilation and Execution

```

gcc hdoc41.c -o hdoc41
./hdoc41

```

Compilation



```

(kali@kali)-[~/Desktop]
$ vi hdoc41.c

(kali@kali)-[~/Desktop]
$ gcc hdoc41.c -o hdoc41

```

Execution

```
(kali㉿kali)-[~/Desktop]
$ ./hdoc41
Enter paragraph 1:
This is a simple paragraph.
Enter paragraph 2:
C programming is fun.
```

Test Case 1

```
6. Exit
Enter your choice: 1
Paragraph 1 - Longest: paragraph, Shortest: a
Paragraph 2 - Longest: programming, Shortest: C
```

Test Case 2

```
6. Exit
Enter your choice: 2
Paragraph 1: This is a simple paragraph.
Paragraph 2: C programming is fun.
```

Test Case 3

```
6. Exit
Enter your choice: 3
Paragraph 1: This Is A Simple Paragraph.
Paragraph 2: C Programming Is Fun.
```

Test Case 4

```
6. Exit
Enter your choice: 4
Paragraph 1 word frequencies:
this = 1
is = 1
a = 1
simple = 1
paragraph = 1
Paragraph 2 word frequencies:
c = 1
programming = 1
is = 1
fun = 1
```

Test Case 5

```
6. Exit
Enter your choice: 1
Paragraph 1 - Longest: paragraph, Shortest: a
Paragraph 2 - Longest: programming, Shortest: C
```

Test Case 6

```
6. Exit
Enter your choice: 9
Invalid choice. Please enter 1 to 6.
```

Test Case 7

0. EXIT

Enter your choice: 6

Exiting program.